Security Audit Report

Concentrated Liquidity Market Maker

Date	March 11, 2026
Project	Concentrated Liquidity Market Maker
Version	1.0

Contents

Disclosure 1
Disclaimer 2
Scope 3
Assessment Overview 4
Assessment Components 5
Executive Summary 6
Code Base 7
Severity Classification 8
Finding Severity Ratings 9
Findings 10
[C 501] Staking Rewards Stolen 11
[C 502] Stolen Pool Identifier FT 15
[H 401] Last epoch staking rewards lost 18
[M 301] Multi-Pool Routing DDOS 22
[I 101] Redundant pool index duplication check 23
[I 102] Optimize Redeemer Lookup 25
[I 103] Redundant Pool FT For Staking UTxO 26
[I 104] Redundant `reference_script` Field Data 27

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Danogo** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **December 26, 2025** to **February 02, 2026**, **Danogo** engaged No Witness Labs to conduct a comprehensive security assessment of the **Concentrated Liquidity Market Maker** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual revision

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

The Concentrated Liquidity Market Maker (CLMM) functions as a decentralized exchange (DEX) where trades occur against liquidity pools within user-defined price ranges. This design delivers greater capital efficiency and higher effective liquidity.

Liquidity providers (LPs) create custom pools—specifying token pairs, price ranges, and fees—with admin authorization. Once active, one can permissionlessly add/remove liquidity or execute swaps. For pools involving ADA token pairs, staking rewards from delegated ADA accrue automatically: they are withdrawn, deposited into the pool permissionlessly each epoch (before price updates), with stake pools selected by the admin.

Code Base

Repository

https://github.com/danogo-finance/clmm/

Commit

e7ae4c00e2bdcc2d6cb6b075b54f311822043cb0

Files Audited

Files	SHA256 Checksum
validators/pool.ak	45c00e59d4c8fc4c357693ebeaa2ed82664dc88dd2f760a863da2136f0a67f3d
validators/staking.ak	8cf4b70a4e38dc892e5a8eebfc88dfe6501f2c368227e86f7afbd20be554f290
lib/sc/sc_fallback.ak	a48c5fb5ad71a89393fdbb1845d1575248589a7626fc38970c6521bbac60eb13
lib/sc/sc_pool.ak	43042ebc801f82eda071b71ad6cf51ec385856fb9ecde10e0619ee8ecd9eb4d3
lib/sc/sc_staking.ak	6b72a6a95b3f138eac6d2a2530376191013ed71918baa89566fdf3b773835bc4
lib/pool/claim_undefined.ak	e13fac6ec703a2e944f5b4874a6751e56e297f8a4366a41e35d06aabe40da83b
lib/pool/create_pool.ak	315abaea8bea1a03bb36d6118ec6da6d7b4cda9fa3918cecee97cc4798a8446a
lib/pool/helpers.ak	446b6c30d04569c0aa6729dd7887fd3dcd52a3379eee19ba526e40df04f809d3
lib/pool/modify_liquidity.ak	ed2ee9b5cd12edc2cd284c5164c0e91710c7732283fbe26ad84d893abc8ac843
lib/pool/swap.ak	fca4565c229e7248c5e713d2827e4b04d5dd571adf7c50a261a30eec74aa66c0
lib/pool/withdraw_platform_fee.ak	655548acfa886c39d2620cc5ebbc4389dcd2b7fb580353c585ace1917ee1b9f7
lib/constants.ak	5752a879cf69b788249ddbdf21e9343c8ff4b999b10825d93c5ae2fc851ef4e1
lib/types.ak	841bdacbecca34d9d13d32030876f938f75e7fa54fad0559014ba1c178457860
lib/utils.ak	e1cb800c64a3a5af6fe01c6cb917661fca7c1dce590b4da5683ed4f50402ae91

Severity Classification

- **Critical**: Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High**: Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium**: Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low**: Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational**: Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

Level	Severity	Status
5	Critical	2
4	High	1
3	Medium	1
2	Low	0
1	Informational	4

Findings

ID	Title	Severity	Status
501	Staking Rewards Stolen	Critical	Resolved
502	Stolen Pool Identifier FT	Critical	Resolved
401	Last epoch staking rewards lost	High	Resolved
301	Multi-Pool Routing DDOS	Medium	Resolved
101	Redundant pool index duplication check	Informational	Resolved
102	Optimize Redeemer Lookup	Informational	Acknowledged
103	Redundant Pool FT For Staking UTxO	Informational	Resolved
104	Redundant `reference_script` Field Data	Informational	Resolved

[C 501] Staking Rewards Stolen

Level	Severity	Status
5	Critical	Resolved

Affected File: `lib/sc/sc_staking.ak`

Description

```
pub fn withdraw(
  rdmr: ByteArray,
  pool_skh: ScriptHash,
  Transaction { redeemers, .. }: Transaction,
) {
  let action_type = builtin.index_bytearray(rdmr, constants.number_1)
  //
  if action_type == constants.rModifyLiquidity || action_type == constants.rSwap {
    utils.require_withdraw_redeemer(redeemers, Script(pool_skh), rdmr)?
  } else {
    fail @"Unsupported action"
  }
}
```

Staking rewards earned should be added to rightful pool's liquidity. This validation is performed by `pool`'s `withdraw` handler for pool indices provided to it via redeemer. `staking`'s `withdraw` handler does not ensure presence of pool utxo's index, whose staking credential it manages, in the redeemer. Allowing withdrawal of a pool's staking rewards without ever spending it and without `pool`'s `withdraw` handler taking staking reward into account. An attacker can steal staking rewards belonging to all pool utxos by carrying out `rModifyLiquidity` or `rSwap` action for a pool without any staking credential.

Similarly, a staking credential could be de-registered or delegated to an inactive stake pool by means of `staking` validator's `publish` handler via `rModifyLiquidity` action, resulting in missed opportunity of earning rewards.

Recommendation

It is recommended to validate presence of pool utxo index in `staking.withdraw`'s redeemer whose staking credential it manages. Similar check should be added to `staking.publish`'s `rModifyLiquidity` action.

```diff
pub fn withdraw(
  rdmr: ByteArray,
+ staking_credentail: Credential,
  pool_skh: ScriptHash,
  Transaction { redeemers, .. }: Transaction,
```

```
) {
+ let pool_in_idx = bultin.index_bytearray(rdmr, constants.number0)
  let action_type = builtin.index_bytearray(rdmr, constants.number_1)

+ expect Input {
+   output: Output {
+     address: Address { stake_credential: pool_in_staking_cre, .. },
+     ..
+   },
+   ..
+ } = utils.list_get(inputs, pool_in_idx)

+ expect staking_credentail == pool_in_staking_cre

+ expect utils.contains_in_idx( // find index based on action type
+   rdmr,
+   pool_in_idx,
+   action_length,
+   step,
+   start,
+   position: constants.number_0,
+ )

  if action_type == constants.rModifyLiquidity || action_type == constants.rSwap {
    utils.require_withdraw_redeemer(redeemers, Script(pool_skh), rdmr)?
  } else {
    fail @"Unsupported action"
  }
}


pub fn publish(
  rdmr: ByteArray,
+ cert: Certificate,
  Transaction { inputs, reference_inputs, extra_signatories, redeemers, .. },
  license_nft_pid: ByteArray,
  license_nft_name: ByteArray,
  pool_skh: ScriptHash,
) {
  let action_type = builtin.index_bytearray(rdmr, constants.number_1)
```

```
  //
  if action_type == constants.rModifyLiquidity {
+ expect UnregisterCredential { staking_credential, .. } = cert
+ let pool_in_idx = bultin.index_bytearray(rdmr, constants.number0)

+ expect Input {
+   output: Output {
+     address: Address { stake_credential: pool_in_staking_cre, .. },
+     ..
+   },
+   ..
+ } = utils.list_get(inputs, pool_in_idx)

+ expect staking_credential == pool_in_staking_cre

+ expect utils.contains_in_idx( // find index
+   rdmr,
+   pool_in_idx,
+   action_length,
+   step,
+   start,
+   position: constants.number_0,
+  )
    utils.require_withdraw_redeemer(redeemers, Script(pool_skh), rdmr)?
  } else if action_type == constants.rCreatePool {
    utils.require_mint_redeemer(redeemers, pool_skh, rdmr)?
  } else {
    let utxo_type = builtin.index_bytearray(rdmr, constants.number_2)
    let license_nft_idx = builtin.index_bytearray(rdmr, constants.number_3)
    //
    utils.has_license_nft(
      inputs,
      reference_inputs,
      extra_signatories,
      license_nft_pid,
      license_nft_name,
      utxo_type,
      license_nft_idx,
    )?
  }
```

```
}
```

Resolution

Resolved in commit hash `9da24aa561204b39d0144269ac94905a04127769`

[C 502] Stolen Pool Identifier FT

Level	Severity	Status
5	Critical	Resolved

Affected File: `lib/sc/sc_staking.ak`

Description

```
pub fn spend(
  rdmr: ByteArray,
  out_ref: OutputReference,
  Transaction { inputs, redeemers, .. },
  pool_skh: ScriptHash,
) {
  let staking_in_idx = builtin.index_bytearray(rdmr, constants.number_0)
  // Input
  let Input { output_reference, .. } = utils.list_get(inputs, staking_in_idx) // <-- staking_in_idx can be 255
  //
  expect output_reference == out_ref
  //
  let action_type = builtin.index_bytearray(rdmr, constants.number_1)

  ..
}
```

`staking` validator's `spend` endpoint allows spending of UTxO with transaction input index of 255, which the protocol considers as `None` value for input index (`constants.none == 255`). In `pool` validator's `rModifyLiquidity` validation when `staking_in_idx == constants.none` it is assumed that staking UTxO is not spent. This assumption could be stemming from two factors, firstly that `staking`'s `spend` would prevent spending of 255 index and secondly, input index of 255 could not be attained. Both of which don't hold true as a transaction with 260 inputs under a single payment credential can be spent with a transaction size of 10 kB. This theoretically allows a protocol transaction of modifying a pool's liquidity and spending it's associated staking UTxO with a input index of 255 within 16 kB of transaction size limit.

Pool UTxO and it's associated staking UTxO both contain the same pool identifier FT. Protocol intends to spend this staking UTxO only when a pool's entire liquidity is withdrawn but this vulnerability allows it to be spent before pool's closure, giving attacker the pool identifier FT. With this FT an attacker can create a fake pool UTxO with an arbitrary `circulating_lp_token` value allowing him to mint actual LP tokens for fraction of a cost and then redeeming them against the original pool UTxO. Thus, allowing him to steal funds from all the pools with ADA token pair.

```
// lib/pool/modify_liquidity.ak

if new_circulating_lp_token == constants.zero {
  expect has_unreg
  //
  return(
    pool_in_idx,
    total_fee_val
      |> assets.add(token_x_pid, token_x_name, platform_fee_x)
      |> assets.add(token_y_pid, token_y_name, platform_fee_y),
    mint_remain
      |> assets.add(pool_skh, pool_nft_name, pool_nft_qty)
      |> assets.add(pool_skh, lp_name, -lp_token_delta_amount),
  )
} else {
  //
  let staking_in_idx =
    builtin.index_bytearray(rdmr, current_idx + constants.number_2)
  // Not allow spend staking when lp token is still circulating
  expect staking_in_idx == constants.none // <-- staking_in_idx with value 255 will pass validation
  //
  // Not allow unregister staking while lp token is still circulating
  expect !token_x_is_ada || !has_unreg

  ...

}
```

Recommendation

Spending of staking UTxO with index 255 should not be allowed.

```
pub fn spend(
  rdmr: ByteArray,
  out_ref: OutputReference,
  Transaction { inputs, redeemers, .. },
  pool_skh: ScriptHash,
) {
  let staking_in_idx = builtin.index_bytearray(rdmr, constants.number_0)
  expect staking_in_idx != constants.none // <-- staking_in_idx cannot be 255
```

```
    // Input
    let Input { output_reference, .. } = utils.list_get(inputs, staking_in_idx)
    //
    expect output_reference == out_ref
    //
    let action_type = builtin.index_bytearray(rdmr, constants.number_1)

    ..

}
```

Resolution

Resolved in commit hash `9da24aa561204b39d0144269ac94905a04127769`

[H 401] Last epoch staking rewards lost

Level	Severity	Status
4	High	Resolved

Affected File: `lib/pool/modify_liquidity.ak`

Description

```
if new_circulating_lp_token == constants.zero {
  expect has_unreg // <-- staking credential required to be deregistered
  //
  return(
    pool_in_idx,
    total_fee_val
      |> assets.add(token_x_pid, token_x_name, platform_fee_x)
      |> assets.add(token_y_pid, token_y_name, platform_fee_y),
    mint_remain
      |> assets.add(pool_skh, pool_nft_name, pool_nft_qty)
      |> assets.add(pool_skh, lp_name, -lp_token_delta_amount),
  )
}
```

Given current epoch is numbered "n", staking rewards belonging to previous epoch (n-1) will be provided in epoch n+1. However, de-registering a stake credential in epoch n results in losing staking rewards for epoch n-1. The protocol enforces the stake credential to be deregistered in the transaction where pool liquidity goes to zero. This results in the second last epoch staking rewards associated with all ADA pools to be lost. These rewards could have either been provided to lp token holder addresses who withdrew in epoch n or claimed by the protocol.

Recommendation

De-registration should not be conducted when protocol liquidity goes to zero. Instead a pool output with pool FT and `NoDatum` should be created with the same staking credential. Using license nft for new pool action `ClaimRewards`, n-1 epoch rewards should be allowed to be withdrawn in n+1 epoch along with spending pool UTxO, burning pool FT and deregistering stake credential.

```diff
// lib/pool/modify_liquidity.ak

if new_circulating_lp_token == constants.zero {
- expect has_unreg // <-- staking credential required to be deregistered
  //
  return(
    pool_in_idx,
```

```
      total_fee_val
        |> assets.add(token_x_pid, token_x_name, platform_fee_x)
        |> assets.add(token_y_pid, token_y_name, platform_fee_y),
      mint_remain
-       |> assets.add(pool_skh, pool_nft_name, pool_nft_qty)
        |> assets.add(pool_skh, lp_name, -lp_token_delta_amount),
    )
  }

  // lib/sc/sc_staking.ak

  pub fn withdraw(
    rdmr: ByteArray,
    pool_skh: ScriptHash,
    staking_credential: Credential,
    Transaction { redeemers, inputs, .. }: Transaction,
  ) {
    let pool_in_idx = builtin.index_bytearray(rdmr, constants.number_0)
    let action_type = builtin.index_bytearray(rdmr, constants.number_1)
    let Input { output: Output { address: pool_in_addr, .. }, .. } =
      utils.list_get(inputs, pool_in_idx)
    //
    expect
      pool_in_addr == Address(Script(pool_skh), Some(Inline(staking_credential)))
    // This Pool input already verified in purpose spend of pool script
-   if action_type == constants.rModifyLiquidity || action_type == constants.rSwap {
+   if action_type == constants.rModifyLiquidity || action_type == constants.rSwap || action_type == constants.rClaimRewards {
      utils.require_withdraw_redeemer(redeemers, Script(pool_skh), rdmr)?
    } else {
      fail @"Unsupported action"
    }
  }

  pub fn publish(
    rdmr: ByteArray,
    Transaction {
      inputs,
      reference_inputs,
      extra_signatories,
```

```
    redeemers,
    outputs,
    ..
  },
  license_nft_pid: ByteArray,
  license_nft_name: ByteArray,
  pool_skh: ScriptHash,
  certificate: Certificate,
) {
  let action_type = builtin.index_bytearray(rdmr, constants.number_1)
  //
- if action_type == constants.rModifyLiquidity {
-   let pool_in_idx = builtin.index_bytearray(rdmr, constants.number_0)
-   let Input { output: Output { address: pool_in_addr, .. }, .. } =
-     utils.list_get(inputs, pool_in_idx)
-   when certificate is {
-     // Only support
-     UnregisterCredential { credential, .. } -> {
-       expect
-         pool_in_addr == Address(Script(pool_skh), Some(Inline(credential)))
-     }
-     _ -> fail @"Invalid certificate for pool staking credential"
-   }
-   utils.require_withdraw_redeemer(redeemers, Script(pool_skh), rdmr)?
- } else if action_type == constants.rCreatePool {
+ if action_type == constants.rCreatePool {
    let pool_ou_idx = builtin.index_bytearray(rdmr, constants.number_3)
    let Output { address: pool_out_addr, .. } =
      utils.list_get(outputs, pool_ou_idx)
    when certificate is {
      // Only support
      RegisterCredential { credential, .. } |
      DelegateCredential { credential, .. } |
      RegisterAndDelegateCredential { credential, .. } -> {
        expect
          pool_out_addr == Address(Script(pool_skh), Some(Inline(credential)))
      }
      _ -> fail @"Invalid certificate for pool staking credential"
    }
    utils.require_mint_redeemer(redeemers, pool_skh, rdmr)?
```

```
  } else {
    let utxo_type = builtin.index_bytearray(rdmr, constants.number_2)
    let license_nft_idx = builtin.index_bytearray(rdmr, constants.number_3)
    //
    utils.has_license_nft(
      inputs,
      reference_inputs,
      extra_signatories,
      license_nft_pid,
      license_nft_name,
      utxo_type,
      license_nft_idx,
    )?
  }
}
```

Resolution

Protocol admin will claim the unentitled staking reward for the last epoch.
Resolved in commit hash `8a21cebec1a60a389ce416df6caa82cc2c9abdb4`

[M 301] Multi-Pool Routing DDOS

Level	Severity	Status
3	Medium	Resolved

Description

The protocol allows for multi-pool (or multi-hop) swaps (or routing). For e.g. there exits pools with following token pairs A/B, B/C, C/D and D/E. A multi-pool swap means swapping token A for token B, using token B to swap for C, C for D, D for E all in the same transaction. This allows a user to obtains token E in exchange of A through intermediate swaps. All involved pools are spent in a single transaction with minimum swap value (or minimum liquidity requirement) being `maximum(min_a_change, min_b_change, min_c_change, ...)`. This drastically reduces the liquidity requirement for carrying out a denial of service attack on multiple pools to the liquidity requirement of a single pool. Preventing users from carrying out swaps or withdrawals from those pools. Additionally, pool's minimum token change values are immutable once set during pool creation, which prevents their appropriate adjustment depending on real time liquidity and scope of multi-pool routing.

Recommendation

Given the shortlived and non-deterministic nature of concentrated liquidity pools, it is difficult to preemptively determine the scope of multi-pool routing for the many pools and consequently their mininum change values. Hence, it is recommended to keep `min_x_change` and `min_y_change` fields of `PoolDatum` modifiable by protocol admin (i.e. license nft holder). This will allow dynamic adjustments to mininum token change values based on a pool's liquidity and it's available routing paths.

Resolution

An admin controlled `swap_fee` is introduced which would be applicable for each pool swap. It will primarily remain zero but updated to a significant value should the protocol come under a DDOS attack.
Resolved in commit hash `8a21cebec1a60a389ce416df6caa82cc2c9abdb4`

[I 101] Redundant pool index duplication check

Level	Severity	Status
1	Informational	Resolved

Affected File: `lib/pool/modify_liquidity.ak`

Description

```aiken
fn verify_pools(
  state: BitSet,
  current_idx: Int,
  total_fee_val: Value,
  mint_remain: Value,
  verify_pool_callback: fn(Int, Value, Value, fn(Int, Value, Value) -> result) ->
    result,
  return: fn(Value, Value) -> result,
) {
  if current_idx < constants.number_3 {
    return(total_fee_val, mint_remain)
  } else {
    let
      pool_in_idx,
      new_total_fee_val,
      new_mint_remain,
    <- verify_pool_callback(current_idx, total_fee_val, mint_remain)
    verify_pools(
      bitset.must_add(state, pool_in_idx), // <-- redundant index duplication check
      current_idx - constants.number_3,
      new_total_fee_val,
      new_mint_remain,
      verify_pool_callback,
      return,
    )
  }
}
```

`bitset.must_add` function ensures that a pool index is not repeated in the redeemer to prevent any unforeseen vulnerabilities. However, each `verify_pools` validation relies on remaining mint quantities which evaluates to zero lp token mint if a pool input is re-validated. Zero lp token mint results in no change in token pair values and thereby failing validation.

```aiken
let expect_pool_change_x =
```

```
    -(-lp_token_delta_amount * pool_in_token_x_real / pool_in_circulating_lp_token)
  //
  // poolChangeY = CEIL(mint.LPToken / pool_in_datum.circulating_lp_token * poolinLPY)
  // poolChangeY = CEIL(mint.LPToken * poolinLPY / pool_in_datum.circulating_lp_token)
  // poolChangeY = -(-mint.LPToken * poolinLPY / pool_in_datum.circulating_lp_token)
  let expect_pool_change_y =
    -(-lp_token_delta_amount * pool_in_token_y_real / pool_in_circulating_lp_token)

  expect or {
    math.abs(expect_pool_change_x) >= min_x_change,
    math.abs(expect_pool_change_y) >= min_y_change,
  }
```

This makes `bitset.must_add` verification redundant.

Recommendation

To remove the redundant validation check.

Resolution

Resolved in commit hash `8a21cebec1a60a389ce416df6caa82cc2c9abdb4`

[I 102] Optimize Redeemer Lookup

Level	Severity	Status
1	Informational	Acknowledged

Description

The protocol frequently requires looking up redeemers associated with script purposes `Withdraw(Script(pool_skh))` or `Mint(pool_skh)` from `redeemers:Pairs<ScriptPurpose, Redeemer>`. It does so by iterating over the pairs, one after another until a matching script purpose if found. This lookup can be made efficient by providing the redeemer's index via dependant redeemer e.g. pool input spending redeemer containing the pool withdraw redeemer's index.

A transaction orders `redeemers` first via redeemer tag and then lexicographically as per script purpose's field: ; 0: spend ; 1: mint ; 2: cert ; 3: reward ; 4: voting ; 5: proposing

Spend -> Mint -> Cert -> Reward -> Voting -> Proposing (per [conway cddl](https://github.com/IntersectMBO/cardano-ledger/blob/master/eras/conway/impl/cddl/data/conway.cddl#L723C1-L728C15)). Within each tag, purposes are sorted lexicographically by the referenced entity — for Spend, by input `(txId, ix)` tuples; for Mint, by `policyId`; for Cert, by certificate index in the transaction body; for Reward, by `StakingCredential` (e.g., first Spend input gets redeemer index 0).

Recommendation

To provide withdraw and mint redeemer indices via dependant redeemers.

[I 103] Redundant Pool FT For Staking UTxO

Level	Severity	Status
1	Informational	Resolved

Description

There exists two pool identifier FTs one present in the pool itself and another locked in `staking` validator UTxO (aka Staking UTxO) managing pool's staking credential. This is unnecessary as staking UTxO does not lock rewards in it, instead they are sent to pool UTxO itself. Also, a pool UTxO's staking credential never changes thereby always pointing to the correct staking script.

Eliminating pool FT from staking UTxO would result in:

1. Reducing validation and transaction size during pool creation and closure.
2. Simplify `rModifyLiquidity` validation and redeemer by eliminating `staking_in_idx`.
3. Simplify `staking`'s `spend` handler by removing `rModifyLiquidity` action.

Recommendation

To update the validators by eliminating dependence on staking UTxO with pool FT.

Resolution

Resolved in commit hash `8a21cebec1a60a389ce416df6caa82cc2c9abdb4`

[I 104] Redundant `reference_script` Field Data

Level	Severity	Status
1	Informational	Resolved

Affected File: `lib/pool/create_pool.ak`

Description

```
expect Output {
  address: Address { payment_credential: staking_payment_cre, .. },
  value: staking_ou_val,
  datum: NoDatum,
  reference_script: Some(staking_ref_script_hash),
} = utils.list_get(outputs, staking_ou_idx)
expect staking_payment_cre == Script(staking_ref_script_hash) // <-- redundant validation
expect pool_stake_cre == Some(Inline(staking_payment_cre))
//
// only ADA and NFT
expect assets.has_nft_strict(staking_ou_val, policy_id, pool_nft_name)
//
expect has_reg(certificates, staking_payment_cre)
//
let expect_staking_ref_script_hash =
  apply_params(
    constants.script_version,
    constants.prefix_staking_sc,
    // params
    license_nft_pid,
    license_nft_name,
    policy_id,
    pool_nft_name,
  )
expect staking_ref_script_hash == expect_staking_ref_script_hash
```

It is unnecessary to persist the required `staking_ref_script_hash` in UTxO's `reference_script` field when it's `payment_credential` can directly be validated against `expect_staking_ref_script_hash`. This minimizes UTxO size and consequently transaction fees.

Recommendation

To remove `staking_ref_script_hash` persisted in `reference_script` field.

Resolution

Resolved in commit hash `8a21cebec1a60a389ce416df6caa82cc2c9abdb4`

End of Report

Copyright © 2026